PROJECT PLAN 29 September 2026

Computation-aware Streaming Speech Translation on a Raspberry Pi 5

Student Fredrik Boglind
Course TinyML, Stockholm University, HT 2026
Instructors Ruipu Peng; Jing Zhan

1. Introduction

The initial approach is an English-to-German cascade of pretrained speech recognition and translation models running locally on a Raspberry Pi 5. Suitable models and inference tools will be selected through an initial literature review and pilot experiment. The aim is to assess how actual computation time affects the quality-latency trade-off when deploying language technology on resource-constrained edge hardware. On-device translation matters when audio must remain local or network connectivity is unavailable, i e, in settings where compute may be limited.

2. Problem Statement

This project is inspired by the IWSLT 2026 simultaneous speech translation track, where systems are ranked by translation quality within latency regimes defined using non-computation-aware LongYAAL, which excludes inference time. The track also evaluates computation-aware latency and specifies a single NVIDIA H100 GPU for Docker-based evaluation [3].

On a CPU-only edge device such as the Raspberry Pi 5, inference time may contribute substantially to the delay experienced by a user. For example, a translation based on 1.5 seconds of audio could require several additional seconds of computation before it becomes available. If processing is consistently slower than incoming speech, delays accumulate throughout the recording. Measuring translation quality together with computation-aware latency is therefore important for assessing practical streaming performance on such devices.

3. Research Questions

1. How does including computation time change the observed quality-latency trade-off across ASR model sizes and audio chunk lengths on the Raspberry Pi 5?
2. Which tested configurations can keep up with incoming speech, and how does their performance compare with the same configurations on a laptop CPU?

4. Methodology

System and scope.
The system will use a cascade of pretrained automatic speech recognition (ASR) and text translation models running locally on a Raspberry Pi 5. Initial candidates include Whisper `tiny.en` or `base.en` for ASR and `Helsinki-NLP/opus-mt-en-de` for English-to-German translation, using faster-whisper and CTranslate2 [1,2].

A literature review and pilot experiment will guide the final selection of models, inference precision and streaming settings. The main experiment will compare approximately four configurations, focusing on one primary factor,such as ASR model size, quantization or audio chunk length, while keeping other settings fixed. The configurations will be selected using separate pilot audio and fixed before the main evaluation. The work will focus on inference and measurement using pretrained models; no model training is planned. The main experiment crosses two ASR model sizes (probably `tiny.en`, `base.en`) with two audio chunk lengths, giving four core configurations. The chunk lengths are chosen during the pilot on separate audio and fixed before the main evaluation.

Considered models:

- Smaller Whisper models (`tiny.en`, `base.en`, `small.en`) for automatic speech recognition (ASR).
- A single bilingual translation model such as `facebook/nllb-200-distilled-600M` or `Helsinki-NLP/opus-mt-e` through CTranslate2 [1,2].

Data

Approximately 30-40 minutes of English audio will be selected, with German translations and English transcripts. The idea is to use recorded talks to simulate translating a live talk into German text. The data will come from the MCIF (Multimodal Crosslingual Instruction Following) dataset [4], released under CC-BY 4.0, which has English transcripts and German reference translations.

Streaming

- Recordings will be replayed at normal speed, without access to future audio.
- Initially a baseline implementation will process successive fixed-length audio chunks, transcribing and translating each chunk and emitting its translation when processing is complete. For shorter chunks, a probable limitation is that fixed boundaries can cut through words or sentences and will require more frequent processing. Longer chunks provide more context but make users wait longer before processing starts.

Measurements.

- Evaluation will follow the IWSLT simultaneous speech translation framework where applicable, using its recommended tools where practical [3,5].
- Translation quality will be assessed using chrF and COMET, with the available transcripts and reference translations [6].
- Latency will be measured both with and without computation time to assess its effect on practical streaming performance.
- Inference and timing measurements will run on the Raspberry Pi. Subsequent scoring and analysis will be performed on the laptop.

The main analysis will compare quality-latency plots and the difference between latency measures. Real-time factor (processing time divided by audio duration, excluding deliberate playback waits) and queue growth will indicate whether processing keeps pace. Peak RAM will be a supporting resource measure; temperature and throttling will be logged to help interpret timing variability. Two complete runs per configuration will follow warm-up, with the same audio and documented decoding and thread settings on both devices.

Optional extension.

- If time permits, an alternative streaming strategy may be compared with the fixed-chunk baseline to investigate its effect on translation quality and latency.
- One possible approach is LocalAgreement, which waits for successive predictions to agree on the beginning of a translation before releasing that part as final output. This extension would focus on the same models and two update intervals, keeping the total grid at no more than eight configurations.

5. Timeline

Period	Milestone
29 Sept.–4 Oct.	Initial literature review; model setup; one complete run from audio to scored output. Confirm runtime, memory use and final scope.
5–11 Oct.	Validate logging and evaluation; fix the core configurations.
12–18 Oct.	Run the Pi and laptop experiments; add LocalAgreement only if the core results are complete and time permits.
19–25 Oct.	Analyse quality, latency and resource measurements; write the report.
26–27 Oct.	Final checks; submit report and source code.

6. Expected Outcomes

The deliverables are a reproducible inference and evaluation pipeline, experiment logs and configurations, and a report comparing the four core configurations. Results will identify the observed quality-latency trade-offs and whether accounting for computation changes the assessment of practical performance on the Pi.

7. Challenges and Risks

Slow inference, software compatibility and incorrect timestamp alignment are risks. A pilot will check the complete pipeline and scoring before large runs. Software versions will be recorded, and identical input and decoding settings will be used for comparisons. The extension will be omitted if necessary.

8. References

[1] SYSTRAN. *faster-whisper*. https://github.com/SYSTRAN/faster-whisper

[2] OpenNMT. *CTranslate2: Transformers and quantization documentation*. Model conversion; quantization.

[3] IWSLT. *Simultaneous Speech Translation Track, 2026*. https://iwslt.org/2026/simultaneous

[4] FBK-MT. *MCIF dataset*. https://huggingface.co/datasets/FBK-MT/MCIF

[5] Polák et al. *OmniSTEval*. https://github.com/pe-trik/OmniSTEval

[6] Unbabel. *wmt22-comet-da model card*. https://huggingface.co/Unbabel/wmt22-comet-da